

Library Management System

Spring Boot JSP CRUD Project Submission

Techsaswata

May 3, 2026

1. Project Overview

This project is a Spring Boot MVC application for managing a small library dataset. The two selected entities are Author and Book. The application supports create, read, and update operations for both entities using JSP pages, Spring MVC controllers, service classes, Spring Data JPA repositories, and an H2 database.

The application also includes sample data population, database constraints, custom inner join query support, exception handling for duplicate values, styled JSP views, and unit tests for repository and service logic.

Technology Stack

Layer	Technology
Backend	Spring Boot 3.3.5
Web MVC	Spring MVC
Views	JSP, JSTL, Expression Language
Persistence	Spring Data JPA, Hibernate
Database	H2 in-memory database
Validation	Jakarta Bean Validation
Testing	JUnit 5, Mockito, Spring Boot Test
Build Tool	Maven

2. Entity Relationship Design

The system has a one-to-many relationship between authors and books.

Entity	Main Fields	Constraints
Author	id, name, email, country	name unique, email unique, required fields
Book	id, title, isbn, genre, publishedYear, author	isbn unique, author required

Relationship:

Author 1 ---- * Book

One author can write many books. Each book belongs to one author. In the database, the books table contains author_id as a foreign key referencing the authors table.

Entity Code

```
@Entity
@Table(name = "authors")
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String name;

    @Column(nullable = false, unique = true, length = 120)
    private String email;

    @Column(nullable = false, length = 80)
    private String country;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL,
        orphanRemoval = true, fetch = FetchType.LAZY)
    private List<Book> books = new ArrayList<>();
}
```

@Entity

```

@Table(name = "books")
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 150)
    private String title;

    @Column(nullable = false, unique = true, length = 20)
    private String isbn;

    @Column(nullable = false, length = 80)
    private String genre;

    @Column(nullable = false)
    private Integer publishedYear;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id", nullable = false)
    private Author author;
}

```

3. Database Population

The tables are generated automatically using JPA and Hibernate. DataSeeder runs on application startup and inserts 10 authors and 10 books.

```

@Component
public class DataSeeder implements CommandLineRunner {
    @Override
    public void run(String... args) {
        if (authorRepository.count() > 0) {
            return;
        }

        List<Author> authors = authorRepository.saveAll(List.of(
            new Author("Aarav Mehta", "aarav.mehta@example.com", "India"),
            new Author("Maya Iyer", "maya.iyer@example.com", "India"),
            new Author("John Carter", "john.carter@example.com", "USA")
            // 10 authors total
        ));

        bookRepository.saveAll(List.of(
            new Book("Spring in Action", "9781617294945", "Technology", 2022, authors.get(0)),
            new Book("Clean Code Notes", "9780132350884", "Programming", 2020, authors.get(1)),
            new Book("Cloud Patterns", "9781492050285", "Technology", 2021, authors.get(2))
            // 10 books total
        ));
    }
}

```

Library Manager

Books Authors Add Book

INNER JOIN RESULT

Books with Author Details

This page uses a custom repository query with an inner join between books and authors.

BOOK	ISBN	GENRE	YEAR	AUTHOR	EMAIL	COUNTRY	ACTION
Spring in Action	9781617294945	Technology	2022	Aarav Mehta	aarav.mehta@example.com	India	Edit
River of Pages	9780000000005	Drama	2018	Carlos Rivera	carlos.rivera@example.com	Spain	Edit
The Silent Library	9780000000004	Mystery	2019	Emily Stone	emily.stone@example.com	UK	Edit
Desert Winds	9780000000009	Adventure	2020	Fatima Khan	fatima.khan@example.com	UAE	Edit
Cloud Patterns	9781492050285	Technology	2021	John Carter	john.carter@example.com	USA	Edit
Northern Lights	9780000000007	Fiction	2017	Liam Brooks	liam.brooks@example.com	Canada	Edit
Clean Code Notes	9780132350884	Programming	2020	Maya Iyer	maya.iyer@example.com	India	Edit
Data Stories	9780000000008	Analytics	2024	Nora Schmidt	nora.schmidt@example.com	Germany	Edit
Paris Letters	9780000000010	Romance	2021	Olivia Martin	olivia.martin@example.com	France	Edit
Tokyo Algorithms	9780000000006	Education	2023	Sakura Tanaka	sakura.tanaka@example.com	Japan	Edit

Figure 1: Books list with joined author details.

Library Manager

Books Authors Add Author

AUTHOR TABLE

Manage Authors

Create and update author records. Books reference authors using a many-to-one relationship.

NAME	EMAIL	COUNTRY	ACTION
Aarav Mehta	aarav.mehta@example.com	India	Edit
Maya Iyer	maya.iyer@example.com	India	Edit
John Carter	john.carter@example.com	USA	Edit
Emily Stone	emily.stone@example.com	UK	Edit
Carlos Rivera	carlos.rivera@example.com	Spain	Edit
Sakura Tanaka	sakura.tanaka@example.com	Japan	Edit
Liam Brooks	liam.brooks@example.com	Canada	Edit
Nora Schmidt	nora.schmidt@example.com	Germany	Edit
Fatima Khan	fatima.khan@example.com	UAE	Edit
Olivia Martin	olivia.martin@example.com	France	Edit

Figure 2: Authors table with seeded and created records.

4. Repository Layer

Both repositories extend `JpaRepository`. The book repository includes a custom inner join query that returns book and author data as a DTO projection.

```
public interface AuthorRepository extends JpaRepository<Author, Long> {
    boolean existsByEmail(String email);
    boolean existsByName(String name);
}

public interface BookRepository extends JpaRepository<Book, Long> {
    boolean existsByIsbn(String isbn);

    @Query("""
        select new com.example.librarymanagement.dto.BookAuthorView(
            b.id, b.title, b.isbn, b.genre, b.publishedYear,
            a.id, a.name, a.email, a.country
        )
        from Book b
    """)
    List<BookAuthorView> findAll();
}
```

```

        inner join b.author a
        order by a.name, b.title
        """)
    List<BookAuthorView> findBooksWithAuthors();
}

```

5. Service Layer

Service classes contain the business logic and connect controllers to repositories. The service layer also checks integrity conditions such as duplicate ISBN and missing author selection.

```

public Book create(Book book) {
    if (bookRepository.existsByIsbn(book.getIsbn())) {
        throw new DataIntegrityViolationException("A book with this ISBN already exists.");
    }

    Long authorId = book.getAuthor() == null ? null : book.getAuthor().getId();
    if (authorId == null) {
        throw new DataIntegrityViolationException("Please select an author for the book.");
    }

    Author author = authorService.findById(authorId);
    book.setAuthor(author);
    return bookRepository.save(book);
}

public Book update(Long id, Book bookDetails) {
    Book book = findById(id);
    Author author = authorService.findById(bookDetails.getAuthor().getId());

    book.setTitle(bookDetails.getTitle());
    book.setIsbn(bookDetails.getIsbn());
    book.setGenre(bookDetails.getGenre());
    book.setPublishedYear(bookDetails.getPublishedYear());
    book.setAuthor(author);

    return bookRepository.save(book);
}

```

6. Controller Layer

Spring MVC controllers handle request mapping, form binding, validation errors, success messages, and redirects.

```

@GetMapping
public String listBooks(Model model) {
    model.addAttribute("bookAuthorRows", bookService.findBooksWithAuthors());
    return "books/list";
}

@PostMapping
public String createBook(@Valid @ModelAttribute("book") Book book,
                        BindingResult bindingResult,
                        Model model,
                        RedirectAttributes redirectAttributes) {
    if (bindingResult.hasErrors()) {
        prepareBookForm(model, "Add Book", "/books");
        return "books/form";
    }

    try {
        bookService.create(book);
        redirectAttributes.addFlashAttribute("successMessage", "Book added successfully.");
        return "redirect:/books";
    } catch (DataIntegrityViolationException ex) {
        bindingResult.reject("integrity", ex.getMessage());
    }
}

```

```

        prepareBookForm(model, "Add Book", "/books");
        return "books/form";
    }
}

```

7. View Layer

The UI is implemented using JSP pages with JSTL and Spring form tags. CSS is used for layout, tables, forms, buttons, alerts, and page structure.

Main JSP pages:

Page	Purpose
books/list.jsp	Displays books with author details using the inner join result
books/form.jsp	Adds and updates books
authors/list.jsp	Displays authors
authors/form.jsp	Adds and updates authors
error.jsp	Displays handled errors

8. Create Operation

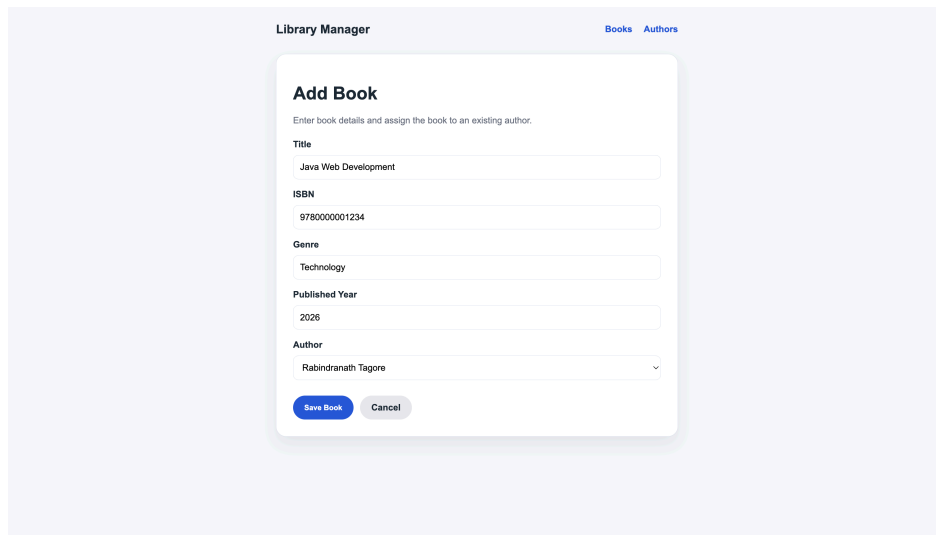
Add Author

Figure 3: Add author form.

NAME	EMAIL	COUNTRY	ACTION
Aarav Mehta	aarav.mehta@example.com	India	Edit
Maya Iyer	maya.iyer@example.com	India	Edit
John Carter	john.carter@example.com	USA	Edit
Emily Stone	emily.stone@example.com	UK	Edit
Carlos Rivera	carlos.rivera@example.com	Spain	Edit
Sakura Tanaka	sakura.tanaka@example.com	Japan	Edit
Liam Brooks	liam.brooks@example.com	Canada	Edit
Nora Schmidt	nora.schmidt@example.com	Germany	Edit
Fatima Khan	fatima.khan@example.com	UAE	Edit
Olivia Martin	olivia.martin@example.com	France	Edit
Rabindranath Tagore	tagore@example.com	India	Edit

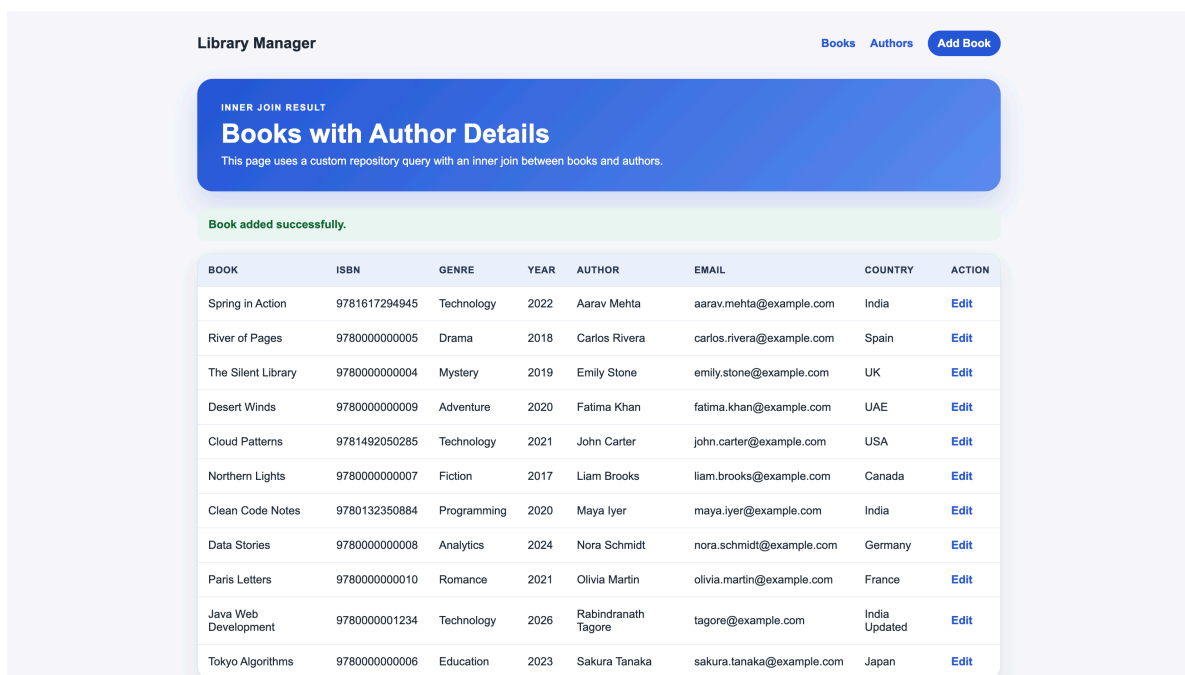
Figure 4: Author added successfully.

Add Book



The image shows a web application interface for a 'Library Manager'. At the top, there are links for 'Books' and 'Authors'. The main content is a modal form titled 'Add Book'. Below the title, a subtitle reads 'Enter book details and assign the book to an existing author.' The form contains several input fields: 'Title' (with the text 'Java Web Development'), 'ISBN' (with '9780000001234'), 'Genre' (with 'Technology'), 'Published Year' (with '2026'), and 'Author' (a dropdown menu showing 'Rabindranath Tagore'). At the bottom of the form are two buttons: 'Save Book' (in blue) and 'Cancel' (in grey).

Figure 5: Add book form with author selection.



The image shows the 'Library Manager' dashboard. At the top, there are links for 'Books', 'Authors', and an 'Add Book' button. Below these links is a blue banner with the text 'INNER JOIN RESULT' and 'Books with Author Details'. Below the banner is a green message box that says 'Book added successfully.' Below this is a table with 8 columns: BOOK, ISBN, GENRE, YEAR, AUTHOR, EMAIL, COUNTRY, and ACTION. The table contains 11 rows of book data. The last row, 'Java Web Development', has 'India Updated' in the COUNTRY column.

BOOK	ISBN	GENRE	YEAR	AUTHOR	EMAIL	COUNTRY	ACTION
Spring in Action	9781617294945	Technology	2022	Aarav Mehta	aarav.mehta@example.com	India	Edit
River of Pages	9780000000005	Drama	2018	Carlos Rivera	carlos.rivera@example.com	Spain	Edit
The Silent Library	9780000000004	Mystery	2019	Emily Stone	emily.stone@example.com	UK	Edit
Desert Winds	9780000000009	Adventure	2020	Fatima Khan	fatima.khan@example.com	UAE	Edit
Cloud Patterns	9781492050285	Technology	2021	John Carter	john.carter@example.com	USA	Edit
Northern Lights	9780000000007	Fiction	2017	Liam Brooks	liam.brooks@example.com	Canada	Edit
Clean Code Notes	9780132350884	Programming	2020	Maya Iyer	maya.iyer@example.com	India	Edit
Data Stories	9780000000008	Analytics	2024	Nora Schmidt	nora.schmidt@example.com	Germany	Edit
Paris Letters	9780000000010	Romance	2021	Olivia Martin	olivia.martin@example.com	France	Edit
Java Web Development	9780000001234	Technology	2026	Rabindranath Tagore	tagore@example.com	India Updated	Edit
Tokyo Algorithms	9780000000006	Education	2023	Sakura Tanaka	sakura.tanaka@example.com	Japan	Edit

Figure 6: Book added successfully.

9. Read Operation

The read operation fetches data from the service layer and binds it to JSP views through the controller model. The `/books` page displays joined book-author data. The `/authors` page displays all authors.

```
@GetMapping
public String listBooks(Model model) {
    model.addAttribute("bookAuthorRows", bookService.findBooksWithAuthors());
    return "books/list";
}
```

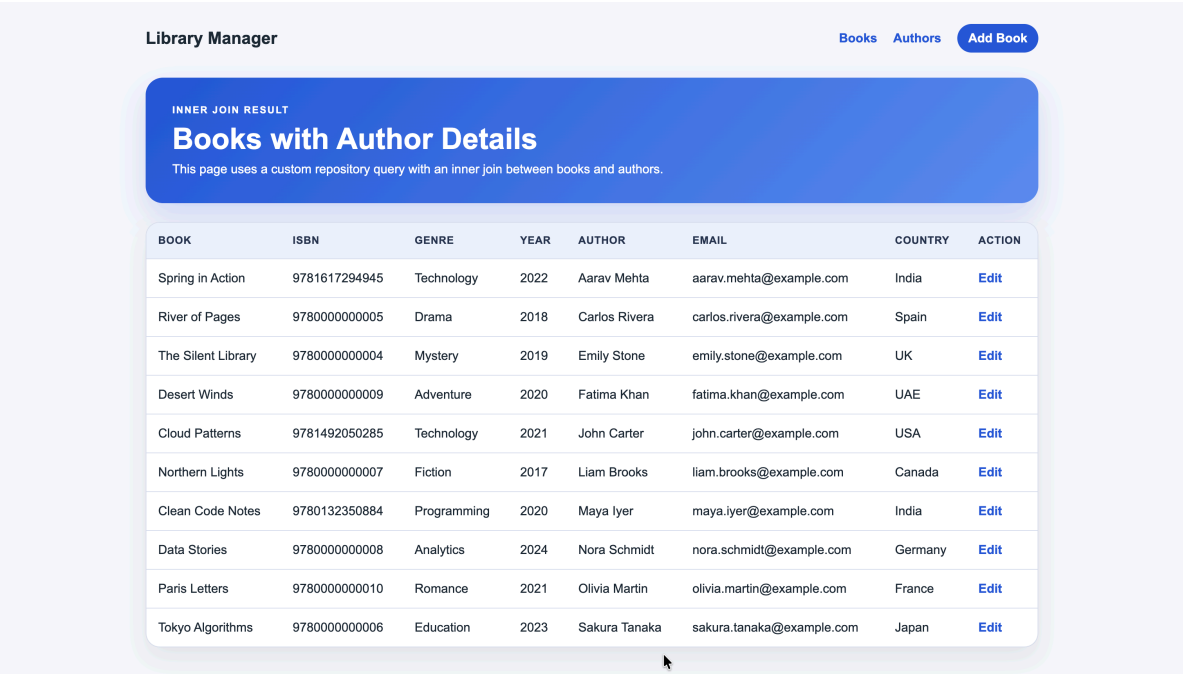


Figure 7: Books read page.

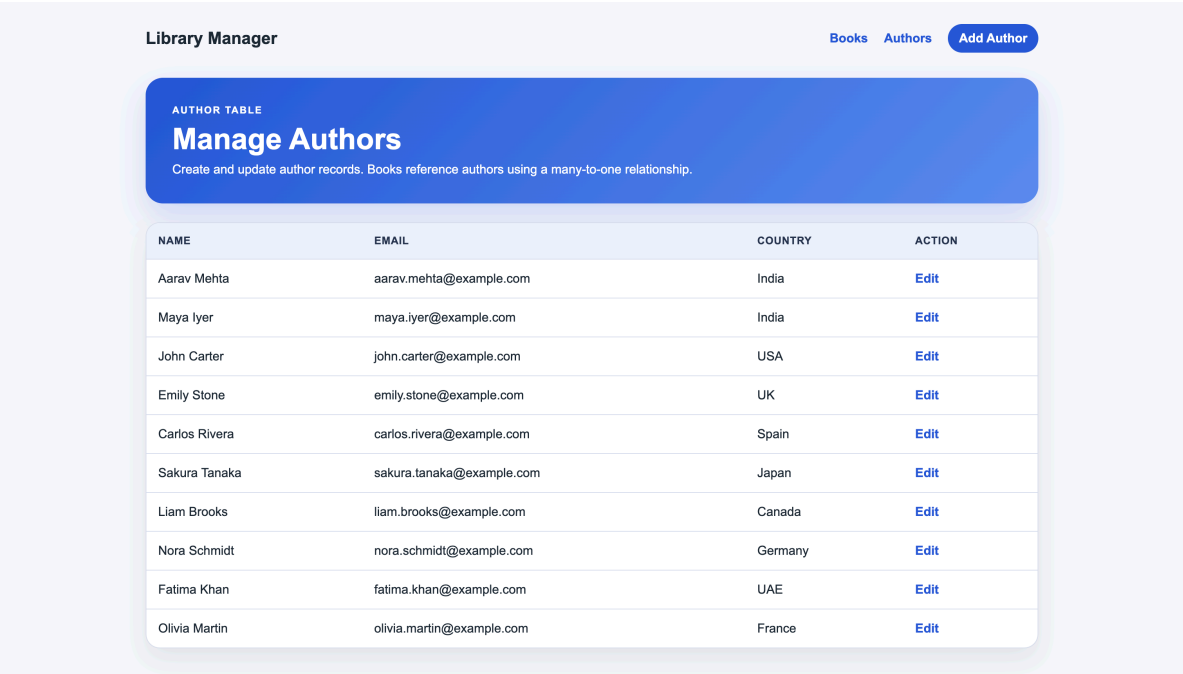


Figure 8: Authors read page.

10. Update Operation

Update Author

Library Manager

BooksAuthors

Update Author

Author name and email are unique, so duplicate values will be rejected.

Name

Rabindranath Tagore

Email

tagore@example.com

Country

India Updated

Save Author

Cancel

Figure 9: Edit author form.

Library Manager

BooksAuthorsAdd Author

AUTHOR TABLE

Manage Authors

Create and update author records. Books reference authors using a many-to-one relationship.

Author updated successfully.

NAME	EMAIL	COUNTRY	ACTION
Aarav Mehta	aarav.mehta@example.com	India	Edit
Maya Iyer	maya.iyer@example.com	India	Edit
John Carter	john.carter@example.com	USA	Edit
Emily Stone	emily.stone@example.com	UK	Edit
Carlos Rivera	carlos.rivera@example.com	Spain	Edit
Sakura Tanaka	sakura.tanaka@example.com	Japan	Edit
Liam Brooks	liam.brooks@example.com	Canada	Edit
Nora Schmidt	nora.schmidt@example.com	Germany	Edit
Fatima Khan	fatima.khan@example.com	UAE	Edit
Olivia Martin	olivia.martin@example.com	France	Edit
Rabindranath Tagore	tagore@example.com	India Updated	Edit

Figure 10: Author updated successfully.

Update Book

Library Manager

BooksAuthors

Update Book

Enter book details and assign the book to an existing author.

Title

Java Web Development

ISBN

9780000001234

Genre

Updated Technology

Published Year

2025

Author

Rabindranath Tagore

Save Book

Cancel

Figure 11: Edit book form.

Library Manager

Books Authors [Add Book](#)

INNER JOIN RESULT

Books with Author Details

This page uses a custom repository query with an inner join between books and authors.

Book updated successfully.

BOOK	ISBN	GENRE	YEAR	AUTHOR	EMAIL	COUNTRY	ACTION
Spring in Action	9781617294945	Technology	2022	Aarav Mehta	aarav.mehta@example.com	India	Edit
River of Pages	9780000000005	Drama	2018	Carlos Rivera	carlos.rivera@example.com	Spain	Edit
The Silent Library	9780000000004	Mystery	2019	Emily Stone	emily.stone@example.com	UK	Edit
Desert Winds	9780000000009	Adventure	2020	Fatima Khan	fatima.khan@example.com	UAE	Edit
Cloud Patterns	9781492050285	Technology	2021	John Carter	john.carter@example.com	USA	Edit
Northern Lights	9780000000007	Fiction	2017	Liam Brooks	liam.brooks@example.com	Canada	Edit
Clean Code Notes	9780132350884	Programming	2020	Maya Iyer	maya.iyer@example.com	India	Edit
Data Stories	9780000000008	Analytics	2024	Nora Schmidt	nora.schmidt@example.com	Germany	Edit
Paris Letters	9780000000010	Romance	2021	Olivia Martin	olivia.martin@example.com	France	Edit
Java Web Development	9780000001234	Updated Technology	2025	Rabindranath Tagore	tagore@example.com	India Updated	Edit
Tokyo Algorithms	9780000000006	Education	2023	Sakura Tanaka	sakura.tanaka@example.com	Japan	Edit

Figure 12: Book updated successfully.

11. Exception Handling

Integrity violations are handled for duplicate author name/email and duplicate book ISBN. The form is shown again with an error message instead of crashing the application.

```
try {
    authorService.create(author);
    redirectAttributes.addFlashAttribute("successMessage", "Author added successfully.");
    return "redirect:/authors";
} catch (DataIntegrityViolationException ex) {
    bindingResult.reject("duplicate", ex.getMostSpecificCause().getMessage());
    return "authors/form";
}
```

Library Manager

Books Authors

Add Author

Author name and email are unique, so duplicate values will be rejected.

An author with this email already exists.

Name

Rabindranath Tagore

Email

tagore@example.com

Country

India

Save Author

Cancel

Figure 13: Duplicate author error handling.

Library Manager Books Authors

Add Book

Enter book details and assign the book to an existing author.

A book with this ISBN already exists.

Title
Duplicate ISBN Book

ISBN
9780000001234

Genre
Test

Published Year
2026

Author
Rabindranath Tagore

[Save Book](#) [Cancel](#)

Figure 14: Duplicate book ISBN error handling.

12. H2 Database Verification

The H2 console was used to confirm table creation, sample data, created records, and the SQL inner join.

← → ↻ ⓘ localhost:8080/h2-console/login.jsp?sessionId=50daab36541b93812f53743b3451d6ce

English Preferences Tools Help

Login

Saved Settings: Generic H2 (Embedded)

Setting Name: Generic H2 (Embedded) [Save](#) [Remove](#)

Driver Class: org.h2.Driver

JDBC URL: jdbc:h2:mem:librarydb

User Name: sa

Password:

[Connect](#) [Test Connection](#)

Figure 15: H2 console login.

Auto commit: ☒ Max rows: 1000 Auto complete: Off Auto select: On

jdbc:h2:mem:librarydb
 AUTHORS
 BOOKS
 INFORMATION_SCHEMA
 Users
 H2 2.2.224 (2023-09-17)

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM AUTHORS;

ID	COUNTRY	NAME	EMAIL
1	India	Aarav Mehta	aarav.mehta@example.com
2	India	Maya Iyer	maya.iyer@example.com
3	USA	John Carter	john.carter@example.com
4	UK	Emily Stone	emily.stone@example.com
5	Spain	Carlos Rivera	carlos.rivera@example.com
6	Japan	Sakura Tanaka	sakura.tanaka@example.com
7	Canada	Liam Brooks	liam.brooks@example.com
8	Germany	Nora Schmidt	nora.schmidt@example.com
9	UAE	Fatima Khan	fatima.khan@example.com
10	France	Olivia Martin	olivia.martin@example.com
11	India Updated	Rabindranath Tagore	tagore@example.com

(11 rows, 3 ms)

Edit

Figure 16: Authors table in H2.

Auto commit: ☒ Max rows: 1000 Auto complete: Off Auto select: On

jdbc:h2:mem:librarydb
 AUTHORS
 BOOKS
 INFORMATION_SCHEMA
 Users
 H2 2.2.224 (2023-09-17)

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM BOOKS;

PUBLISHED_YEAR	AUTHOR_ID	ID	ISBN	GENRE	TITLE
2022	1	1	9781617294945	Technology	Spring in Action
2020	2	2	9780132350884	Programming	Clean Code Notes
2021	3	3	9781492050285	Technology	Cloud Patterns
2019	4	4	9780000000004	Mystery	The Silent Library
2018	5	5	9780000000005	Drama	River of Pages
2023	6	6	9780000000006	Education	Tokyo Algorithms
2017	7	7	9780000000007	Fiction	Northern Lights
2024	8	8	9780000000008	Analytics	Data Stories
2020	9	9	9780000000009	Adventure	Desert Winds
2021	10	10	9780000000010	Romance	Paris Letters
2025	11	11	9780000001234	Updated Technology	Java Web Development

(11 rows, 1 ms)

Edit

Figure 17: Books table in H2.

SQL join used for verification:

```
SELECT b.TITLE, b.ISBN, a.NAME AS AUTHOR_NAME, a.EMAIL
FROM BOOKS b
INNER JOIN AUTHORS a ON b.AUTHOR_ID = a.ID;
```

jdvc:h2:mem:librarydb
 AUTHORS
 BOOKS
 INFORMATION_SCHEMA
 Users
 H2 2.2.224 (2023-09-17)

Run | Run Selected | Auto complete | Clear | SQL statement:
 SELECT b.TITLE, b.ISBN, a.NAME AS AUTHOR_NAME, a.EMAIL
 FROM BOOKS b
 INNER JOIN AUTHORS a ON b.AUTHOR_ID = a.ID;

SELECT b.TITLE, b.ISBN, a.NAME AS AUTHOR_NAME, a.EMAIL
 FROM BOOKS b
 INNER JOIN AUTHORS a ON b.AUTHOR_ID = a.ID;

TITLE	ISBN	AUTHOR_NAME	EMAIL
Spring in Action	9781617294945	Aarav Mehta	aarav.mehta@example.com
Clean Code Notes	9780132350884	Maya Iyer	maya.iyer@example.com
Cloud Patterns	9781492050285	John Carter	john.carter@example.com
The Silent Library	9780000000004	Emily Stone	emily.stone@example.com
River of Pages	9780000000005	Carlos Rivera	carlos.rivera@example.com
Tokyo Algorithms	9780000000006	Sakura Tanaka	sakura.tanaka@example.com
Northern Lights	9780000000007	Liam Brooks	liam.brooks@example.com
Data Stories	9780000000008	Nora Schmidt	nora.schmidt@example.com
Desert Winds	9780000000009	Fatima Khan	fatima.khan@example.com
Paris Letters	9780000000010	Olivia Martin	olivia.martin@example.com
Java Web Development	9780000001234	Rabindranath Tagore	tagore@example.com

(11 rows, 0 ms)

Figure 18: H2 inner join result.

13. Testing

Repository and service methods are tested using JUnit, Mockito, and Spring Boot test support.

Test coverage includes:

- Repository custom inner join method.
- Repository ISBN existence check.
- Service duplicate author email validation.
- Service duplicate book ISBN validation.
- Author update service method.
- Book update service method.

@DataJpaTest

```
class BookRepositoryTest {
    @Test
    void findBooksWithAuthorsReturnsInnerJoinRows() {
        Author author = authorRepository.save(
            new Author("Test Author", "test.author@example.com", "India"));
        bookRepository.save(
            new Book("Test Book", "97800000000999", "Education", 2024, author));

        List<BookAuthorView> rows = bookRepository.findBooksWithAuthors();

        assertThat(rows).hasSize(1);
        assertThat(rows.get(0).title()).isEqualTo("Test Book");
        assertThat(rows.get(0).authorName()).isEqualTo("Test Author");
    }
}
```

```
[techassata@X-DIABLO-X SGA-3may % mvn clean test]
[INFO] Scanning for projects...
[INFO]
[INFO] -----[ com.example:library-management ]-----
[INFO] Building library-management 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ war ]-----
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ library-management ---
[INFO] Deleting /Users/techassata/Downloads/SGA-3may/target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ library-management ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.10.0:compile (default-compile) @ library-management ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 13 source files with javac [debug parameters release 17] to target/classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ library-management ---
[INFO] skip non existing resourceDirectory /Users/techassata/Downloads/SGA-3may/src/test/resources
[INFO]
[INFO] --- compiler:3.10.0:testCompile (default-testCompile) @ library-management ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 3 source files with javac [debug parameters release 17] to target/test-classes
[INFO]
[INFO] --- surefire:3.0.0:test (default-test) @ library-management ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] ----- T E S T S -----
[INFO]
[INFO] Running com.example.librarymanagement.repository.BookRepositoryTest
08:49:49.442 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils : Could not detect default configuration classes for test class [com.example.librarymanagement.repository.BookRepositoryTest]: BookRepositoryTest does not declare any static, non-private, non-final, nested classes annotated with @Configuration.
08:49:49.450 [main] INFO org.springframework.boot.test.context.SpringBootContextBootstrapper : Found @SpringBootConfiguration com.example.librarymanagement.LibraryManagementApplication for test class com.example.librarymanagement.repository.BookRepositoryTest

  ____
 /  _ \
/  / \
/   \
/____\

:: Spring Boot ::
(v3.3.0)

2024-05-03T08:05:18.094+05:30 INFO 35123 --- [library-management] [
-3may] main c.e.l.repository.BookRepositoryTest : Starting BookRepositoryTest using Java 23.0.2 with PID 35123 (started by techassata in /Users/techassata/Downloads/SGA-3may)
2024-05-03T08:05:18.094+05:30 INFO 35123 --- [library-management] [
-3may] main c.e.l.repository.BookRepositoryTest : No active profile set, falling back to 1 default profile: "default"
2024-05-03T08:05:18.211+05:30 INFO 35123 --- [library-management] [
-3may] main s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2024-05-03T08:05:18.227+05:30 INFO 35123 --- [library-management] [
-3may] main s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 13 ms. Found 2 JPA repository interfaces.
2024-05-03T08:05:18.244+05:30 INFO 35123 --- [library-management] [
-3may] main o.s.j.d.e.EmbeddedDatabaseFactory : Replacing 'dataSource' DataSource bean with embedded version
2024-05-03T08:05:18.285+05:30 INFO 35123 --- [library-management] [
-3may] main o.s.j.d.e.EmbeddedDatabaseFactory : Starting embedded database: url=jdbc:h2:mem:e971f8da-678c-4844-86a1-b6ea73738d6;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=fa
l
2024-05-03T08:05:18.383+05:30 INFO 35123 --- [library-management] [
-3may] main o.hibernate.jpa.internal.util.LogHelper : HHN000204: Processing PersistenceUnitInfo [name: default]
2024-05-03T08:05:18.483+05:30 INFO 35123 --- [library-management] [
-3may] main org.hibernate.Version : HHN000412: Hibernate ORM core version 6.5.3.Final
2024-05-03T08:05:18.434+05:30 INFO 35123 --- [library-management] [
-3may] main o.h.c.internal.SessionFactoryInitiator : HHN000020: Second-level cache disabled
2024-05-03T08:05:18.528+05:30 INFO 35123 --- [library-management] [
-3may] main o.e.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup; ignoring JPA class transformer
2024-05-03T08:05:18.638+05:30 INFO 35123 --- [library-management] [
-3may] main o.h.e.t.j.p.i.JtaPlatformInitiator : HHN000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)

Hibernate:
drop table if exists authors cascade
Hibernate:
drop table if exists books cascade
Hibernate:
create table authors (
  id bigint generated by default as identity,
  country varchar(80) not null,
  name varchar(100) not null unique,
  email varchar(120) not null unique,
  primary key (id)
)
Hibernate:
create table books (
  published_year integer not null check ((published_year<2100) and (published_year>1000)),
  author_id bigint not null,
  id bigint generated by default as identity,
  genre varchar(20) not null,
  isbn varchar(13) not null unique,
  title varchar(200) not null unique
)
Hibernate:
insert
into
authors
(country, email, name, id)
values
(?, ?, ?, default)
Hibernate:
insert
into
books
(author_id, genre, isbn, published_year, title, id)
values
(?, ?, ?, ?, ?, default)
Hibernate:
select
  bl_0.id
from
  books bl_0
where
  bl_0.isbn=?
fetch
first 7 rows only
Hibernate:
insert
into
authors
(country, email, name, id)
values
(?, ?, ?, default)
Hibernate:
insert
into
books
(author_id, genre, isbn, published_year, title, id)
values
(?, ?, ?, ?, ?, default)
Hibernate:
select
  bl_0.id,
  bl_0.title,
  bl_0.isbn,
  bl_0.genre,
  bl_0.published_year,
  bl_0.author_id,
  al_0.name,
  al_0.email,
  al_0.country
from
  books bl_0
join
  authors al_0
on al_0.id=bl_0.author_id
order by
  al_0.name,
  bl_0.title
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.623 s -- in com.example.librarymanagement.repository.BookRepositoryTest
[INFO] Running com.example.librarymanagement.service.BookServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.897 s -- in com.example.librarymanagement.service.BookServiceTest
[INFO] Running com.example.librarymanagement.service.AuthorServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.029 s -- in com.example.librarymanagement.service.AuthorServiceTest
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.131 s
[INFO] Finished at: 2024-05-03T08:05:11+05:30
[INFO] -----
```

Figure 19: Maven test execution result.

```
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
WARNING: A Java agent has been loaded dynamically (/Users/techassata/.m2/repository/net/bytebuddy/byte-buddy-agent/1.14.19/byte-buddy-agent-1.14.19.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceAgent for more information
WARNING: Dynamic loading of agents will be disabled by default in a future release
Hibernate:
insert
into
authors
(country, email, name, id)
values
(?, ?, ?, default)
Hibernate:
insert
into
books
(author_id, genre, isbn, published_year, title, id)
values
(?, ?, ?, ?, ?, default)
Hibernate:
select
  bl_0.id
from
  books bl_0
where
  bl_0.isbn=?
fetch
first 7 rows only
Hibernate:
insert
into
authors
(country, email, name, id)
values
(?, ?, ?, default)
Hibernate:
insert
into
books
(author_id, genre, isbn, published_year, title, id)
values
(?, ?, ?, ?, ?, default)
Hibernate:
select
  bl_0.id,
  bl_0.title,
  bl_0.isbn,
  bl_0.genre,
  bl_0.published_year,
  bl_0.author_id,
  al_0.name,
  al_0.email,
  al_0.country
from
  books bl_0
join
  authors al_0
on al_0.id=bl_0.author_id
order by
  al_0.name,
  bl_0.title
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.623 s -- in com.example.librarymanagement.repository.BookRepositoryTest
[INFO] Running com.example.librarymanagement.service.BookServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.897 s -- in com.example.librarymanagement.service.BookServiceTest
[INFO] Running com.example.librarymanagement.service.AuthorServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.029 s -- in com.example.librarymanagement.service.AuthorServiceTest
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.131 s
[INFO] Finished at: 2024-05-03T08:05:11+05:30
[INFO] -----
```

Figure 20: Maven test execution result continued.

14. Challenges Faced

1. Configuring JSP with Spring Boot 3 was not automatic because embedded Tomcat does not compile JSP pages unless the Jasper dependency is present, and JSTL now uses Jakarta namespaces. The issue was handled by adding tomcat-embed-jasper, Jakarta JSTL API, and the GlassFish JSTL runtime, then updating JSP taglib declarations to jakarta.tags.core.
2. The list page needed book and author data together. Rendering entities directly could create lazy-loading problems in JSP and would also expose more entity structure than required. This was handled with a repository-level JPQL inner join that returns a DTO projection, so the JSP receives only the fields needed for display.
3. The book form submits only the selected author's id, not a fully loaded author object. Saving that object directly can lead to invalid references or detached entity behavior. The service layer resolves the selected Author from the database first and then attaches it to the Book before saving.

4. Uniqueness had to be enforced at more than one level. Service checks give clearer feedback for common duplicate cases, while database unique constraints still protect the data if two requests try to insert the same email, name, or ISBN. Controllers catch `DataIntegrityViolationException` and return the user to the same form with an error message.

15. Conclusion

The project implements a complete Spring Boot MVC CRUD application for two related entities. It includes JPA table creation, sample data population, create/read/update operations, JSP views, CSS styling, repository and service layers, custom inner join query support, exception handling, and unit tests.

16. GitHub URL

GitHub URL: [Add GitHub repository URL after pushing the project]